```
def MyClass.a_class_method; end

class MyClass
  def self.another_class_method; end
end

class MyClass
  class << self
    def yet_another_class_method; end
  end
end
```

The first syntax is usually frowned upon by expert Rubyists because it duplicates the class name, making it more difficult to refactor. The second syntax takes advantage of the fact that self in the class definition is the class itself. The third syntax is the trickiest one: the code opens the singleton class and defines the method in there. This last syntax acknowledges the singleton class explicitly, so it will win you some street cred in Ruby circles.

Singleton Classes and instance_eval()

Now that you know about singleton classes, you can also fill in one missing snippet of knowledge about the instance_eval method. In *class_eval()*, on page 107, you learned that instance_eval changes self, and class_eval changes both self and the current class. However, instance_eval also changes the current class; it changes it to the *singleton class* of the receiver. This example uses instance_eval to define a *Singleton Method (114)*:

class_definitions/instance_eval.rb

```
s1, s2 = "abc", "def"

s1.instance_eval do
  def swoosh!; reverse; end
end

s1.swoosh!                # => "cba"
s2.respond_to?(:swoosh!)  # => false
```

You'll rarely, if ever, see instance_eval used purposefully to change the current class, as in the example above. The standard meaning of instance_eval is this: "I want to change self."

Class Attributes

Bill's detailed explanations have left you a bit perplexed. "Okay," you say, "I can see how singleton classes are useful to understanding the object model. But how do I use them in practice?"